

Checkpoint 3 — Raw Event Schema Design

Amen Bank — Real-Time Anomaly Detection Platform

Internship Project Documentation

June 2, 2026

1 Purpose

This checkpoint captures the design of the **raw event schema** that flows into the **raw-events** RedPanda topic and is consumed by the Enrichment Service. It defines the structural contract for the very first data shape entering the streaming pipeline.

This document builds on Checkpoint 2.1 (architecture for Amen Bank scope) and resolves the schema design left open at the end of that session.

2 Design Decision — Schema Structure

2.1 Options Considered

Option	Description	Trade-off
A — Flat with nullables	One unified schema with all fields, operation-specific ones nullable.	Simple but schema gets polluted as operations grow; messy validation.
B — One topic per operation	Four separate topics: raw-retrait , raw-versement , raw-virement , raw-cheque .	Clean per-operation schemas but breaks cross-operation client ordering (critical for LSTM), multiplies operational complexity.
C — Tagged-union envelope	Common envelope + nested operation-specific payload, discriminated by operation_type .	Slight validation complexity but preserves single-topic ordering, clean schema evolution, canonical event-driven pattern.

2.2 Decision: Option C — Tagged-Union Envelope

Rationale:

- **Preserves client temporal ordering.** A single **raw-events** topic partitioned by **client_id** keeps all operations of one client on the same partition in order — essential for the LSTM sequence model.
- **Single consumer group** in the Enrichment Service: simpler ops, monitoring, scaling.
- **Schema evolution** is clean: adding a 5th operation type requires only a new payload variant, no new topic, no new consumer.
- This is the canonical pattern in production event-driven banking systems (CloudEvents, ISO 20022).

3 Core Design Principles

Two principles emerged during the design discussion and apply across all payloads:

Principle 1 — Raw events contain only raw facts

A raw event records *what happened at the moment of the transaction*. It does not contain statistical aggregations (**mean**, **std**, percentiles) or properties of distributions. Those belong to **profiles** (hot/cold) and are computed by the Enrichment Service when it builds the feature vector.

Principle 2 — No derived or lagging fields

A raw event must not contain fields that are computed from history (e.g., **first_time_emitter**) or that become known only after the event (e.g., **is_bounced**, which is determined at check clearing time). Such fields belong elsewhere in the pipeline:

- Derived from history → computed by Enrichment Service.
- Lagging facts → delivered as a separate downstream event (e.g., **check_returned**).

4 Envelope Schema (common to all operations)

Field	Type	Description
<code>event_id</code>	UUID	Unique identifier for the event itself. Used for idempotency, deduplication, and audit trail.
<code>client_id</code>	String	The human/legal entity the operation is for. Used as partition key and for profile lookup.
<code>account_id</code>	String	The specific account affected. A client may have multiple accounts (checking, savings, business) with distinct normal patterns.
<code>employee_id</code>	String	The teller who processed the operation. Used for employee profile lookup and dual-actor anomaly detection.
<code>branch_id</code>	String	The physical agency where the operation occurred.
<code>timestamp</code>	ISO 8601 date-time	When the operation occurred.
<code>amount</code>	Decimal	Monetary amount in the operation.
<code>currency</code>	String (ISO 4217)	Currency code (e.g., TND , EUR , USD).
<code>channel</code>	Enum	For now, teller only (per supervisor scope). Reserved for future expansion (GAB, online).
<code>operation_type</code>	Enum	Discriminator for the payload union: retrait versement virement remise_cheque .

payload	Object	Operation-specific data; structure determined by <code>operation_type</code> .
---------	--------	--

5 Payload Schemas

5.1 Retrait (Cash Withdrawal)

The leanest payload — the envelope already captures most of the operation.

Field	Type	Description
<code>id_verified</code>	Boolean	Whether the teller verified the client's CIN. Systematic skipping is itself an employee-side anomaly signal.
<code>is_third_party</code>	Boolean	Whether the person physically withdrawing the cash differs from the account holder. Directly addresses <i>retrait usurpé par un tiers</i> .
<code>third_party_id</code>	String, nullable	Identifier of the third party if applicable (e.g., power of attorney, authorized agent).

5.2 Versement (Cash Deposit)

Field	Type	Description
<code>is_third_party_depositor</code>	Boolean	Whether the depositor differs from the account holder.
<code>depositor_id</code>	String, nullable	Identifier of the depositor if third party. Repeated third-party deposits into the same account can indicate funneling.
<code>source_of_funds</code>	String, nullable	Declared origin of funds (salary, property sale, business income, inheritance, etc.). Required only above the regulatory threshold ($\approx 10,000$ DT).
<code>source_verified</code>	Boolean	Whether the teller verified the declared source.

5.3 Virement (Bank Transfer)

Field	Type	Description
<code>beneficiary_account</code>	String (RIB/IBAN)	The receiving account identifier.
<code>beneficiary_name</code>	String	The recipient's name as recorded. Name/RIB mismatch is a fraud signal.
<code>transfer_motif</code>	String, nullable	Free-text reason for the transfer (<i>libellé</i>).

<code>is_internal_transfer</code>	Boolean	Whether the destination account is at Amen Bank (internal, reversible) or at another bank (external, higher risk). Derivable from the RIB prefix but stored explicitly for clarity.
-----------------------------------	---------	---

5.4 Remise de Chèques (Check Deposit)

Granularity decision: one event per check, not one per remise. Each check is an independently scorable transaction (a single suspicious check inside a clean remise must still be flagged). The shared `remise_id` preserves the grouping context for features such as “number of checks in this remise”.

Field	Type	Description
<code>remise_id</code>	UUID	Shared identifier across all checks deposited in the same remise. Enables grouping features at enrichment time.
<code>check_number</code>	String	Serial number printed on the check. Used to detect duplicate deposits.
<code>emitter_account</code>	String (RIB)	The account against which the check is drawn.
<code>emitter_name</code>	String	The name of the entity that issued the check.
<code>emitter_bank</code>	String	Identifier of the issuing bank. Internal vs. external impacts risk profile.
<code>check_date</code>	Date	Date written on the check itself. Distinct from the deposit timestamp; a stale check is a red flag.
<code>is_endorsed_to_depositor</code>	Boolean	Whether the check was originally made out to someone else and endorsed to the depositor. Maps directly to <i>endossement à un tiers</i> .

6 Example Event Instances

6.1 Versement above the AML threshold

```
{
  "event_id": "8f3c2a1b-...",
  "client_id": "CL-00482",
  "account_id": "ACC-19273",
  "employee_id": "EMP-117",
  "branch_id": "AGY-TUN-12",
  "timestamp": "2026-06-02T10:42:31Z",
  "amount": 14500.00,
  "currency": "TND",
  "channel": "teller",
  "operation_type": "versement",
  "payload": {
    "is_third_party_depositor": false,
    "depositor_id": null,

```

```

    "source_of_funds": "property_sale",
    "source_verified": true
  }
}

```

6.2 One check inside a multi-check remise

```

{
  "event_id": "c41f...",
  "client_id": "CL-00211",
  "account_id": "ACC-08841",
  "employee_id": "EMP-203",
  "branch_id": "AGY-SFX-04",
  "timestamp": "2026-06-02T11:15:02Z",
  "amount": 3200.00,
  "currency": "TND",
  "channel": "teller",
  "operation_type": "remise_cheque",
  "payload": {
    "remise_id": "f9a8...",
    "check_number": "CHK-7723019",
    "emitter_account": "07 012 0000123456789 12",
    "emitter_name": "STE EXAMPLE SARL",
    "emitter_bank": "BIAT",
    "check_date": "2026-05-28",
    "is_endorsed_to_depositor": false
  }
}

```

7 Open Questions for Future Checkpoints

1. **Channel scope.** Supervisor confirmation pending on whether GAB (ATM) withdrawals can be added to scope alongside teller operations.
2. **Lagging events.** Whether `check_returned` (NSF/bounce) events flow back into the system as a separate event type to update client/emitter profiles.
3. **Serialization format.** JSON for the prototype is fine; for production, Avro or Protobuf with schema registry would enforce the tagged-union contract.

8 Next Step

Design the **client profile** and **employee profile** schemas (hot in Redis, cold in PostgreSQL), defining exactly what statistics and features each one stores and how the Enrichment Service uses them to produce the feature vector for the model.